

TechMind Model v1.1.0

Guía de integración para el equipo de Backend

Objetivo. Desplegar la nueva versión del modelo sin modificar su artefacto ni su política operacional, manteniendo compatibilidad con la interfaz REST 1.0.0.

Componente	Versión / valor
Package	1.1.0
Modelo	1.1.0
Interfaz REST	1.0.0
Arquitectura	TF-IDF Word + Char 3-6 + SGDClassifier optimizado
Clases	backend, cloud, datascience, frontend
Features	30,000 Word + 30,000 Char = 60,000

1. Artefacto e integridad

Archivo a desplegar: techmind_model_v1.1.0.zip.

SHA-256 del modelo aprobado:

```
756b2577e731336ead95852ee1d8d752408762478a23d0bbf42cc9537e136ff6
```

- Validar manifest.json antes del despliegue.
- Verificar también el SHA-256 final del ZIP que acompañe la entrega.
- No modificar techmind_modelo_final.joblib.
- No recalibrar umbrales ni alterar la configuración de inferencia desde backend.

2. Archivos que no deben modificarse

Estos archivos forman parte del artefacto aprobado y deben tratarse como inmutables:

```
artifacts/techmind_modelo_final.joblib
artifacts/techmind_modelo_final_metadata.json
artifacts/configuracion_inferencia.json
artifacts/contrato_inferencia.json
techmind/predictor.py
```

Cambiar stopwords, TF-IDF, n-gramas, SGDClassifier, margen de revisión o umbral de cobertura rompería la correspondencia con el modelo evaluado.

3. Endpoints disponibles

Método	Ruta	Uso
GET	/	Información raíz de la API
GET	/health	Disponibilidad y salud del modelo
GET	/model-info	Versión, arquitectura, límites y umbrales
POST	/predict	Clasificación de contenido técnico
GET	/docs	Swagger UI
GET	/redoc	ReDoc

Secuencia de arranque recomendada

1) Iniciar FastAPI. 2) Consultar GET /health. 3) Enviar tráfico a /predict solo cuando responda HTTP 200 con ready: true. Un HTTP 503 indica que el predictor todavía no está disponible.

4. Validaciones mínimas después del despliegue

GET /health debe responder HTTP 200 y mostrar, como mínimo:

```
{
  "status": "ok",
  "ready": true,
  "api_version": "1.0.0",
  "word_features": 30000,
  "char_features": 30000,
  "total_features": 60000
}
```

GET /model-info debe confirmar:

Campo	Valor esperado
api_version	1.0.0
model_version	1.1.0
word_features	30000
char_features	30000
total_features	60000
limits.max_batch_size	500
limits.max_characters_per_document	50000

5. Contrato de POST /predict

La forma del request se mantiene compatible con la versión anterior:

```
{
  "textos": ["Texto técnico que se desea clasificar."],
  "incluir_explicacion": true,
  "top_n_explicacion": 8,
  "top_k": 4
}
```

Prueba de regresión recomendada

Texto: "Este contenido explica cómo crear una API REST con Spring Boot y Java, incluyendo el uso de controladores, servicios y repositorios."

Resultado esperado: categoria_predicha = backend, estado = aceptada y prediccion_utilizable = true.

6. Cómo interpretar la respuesta

estado	Uso recomendado	Campos clave
aceptada	Puede usarse automáticamente.	requiere_revision=false; prediccion_utilizable=true
revision	Existe clasificación, pero requiere revisión humana.	requiere_revision=true; prediccion_utilizable=true
rechazada	No usar como resultado confiable.	prediccion_utilizable=false

Importante: puntuacion_ganadora, puntuacion_segunda y margen_decision son scores del clasificador, no probabilidades. No convertir un margen de 0.98 en 98%. La API declara margin_is_probability=false.

7. Cobertura Word + Char en v1.1.0

La cobertura se calcula con la suma de características activas de ambas ramas. No usar únicamente word_features_activas para decidir si una entrada tiene cobertura.

```
word_features_activas = 0
char_features_activas = 215
features_activas_total = 215
```

=> Hay cobertura y la predicción puede ser válida.

El campo `terminos_activos` se conserva por compatibilidad con la interfaz anterior y representa la cobertura total. Para nuevas integraciones se recomienda usar `features_activas_total`.

La advertencia "La cobertura depende únicamente de n-gramas de caracteres." es informativa. No implica automáticamente revisión o rechazo; el campo autoritativo es estado.

8. Explicabilidad

Cuando `incluir_explicacion=true`, la respuesta puede contener `positive_terms`, `negative_terms` y `differential_terms`. Cada feature puede incluir `feature_type=word` o `feature_type=char`.

```
{
  "term": "spring",
  "feature_type": "char",
  "tfidf": 0.0759,
  "coefficient": 0.4744,
  "contribution": 0.0360
}
```

Las contribuciones describen el comportamiento matemático del modelo y no deben presentarse como causalidad. Si la aplicación no necesita explicación, usar `incluir_explicacion=false`.

9. Categorías admitidas

backend, cloud, datascience y frontend. La interfaz puede mostrar etiquetas visuales distintas, pero se recomienda conservar el valor técnico original.

10. Checklist de aceptación para Backend

- ☐ ZIP validado con `manifest.json`.
- ☐ SHA-256 del modelo coincide con el valor aprobado.
- ☐ `GET /health -> HTTP 200` y `ready=true`.
- ☐ `GET /model-info -> model_version=1.1.0`.
- ☐ `word_features=30000`, `char_features=30000`, `total_features=60000`.
- ☐ `POST /predict -> HTTP 200`.
- ☐ Caso Spring Boot corto -> `categoria_predicha=backend`.
- ☐ Caso Spring Boot corto -> `prediccion_utilizable=true`.
- ☐ No interpretar `margen_decision` como probabilidad.
- ☐ No usar `word_features_activas=0` como sinónimo de falta de cobertura.
- ☐ Respetar estado, `requiere_revision` y `prediccion_utilizable`.

11. Criterio de entrega

El paquete fue validado con pruebas del predictor, pruebas FastAPI y smoke test independiente. El equipo de backend debe integrar y desplegar el paquete sin alterar el artefacto de ML ni su configuración operacional.

Resumen de versiones: API 1.0.0 - Modelo 1.1.0 - Package 1.1.0.